SRS

Engineering is systematic, disciplined, quantifiable approac

Software-
* Engineered, not manufactured
* doesn't wear out
* Change is inevitable

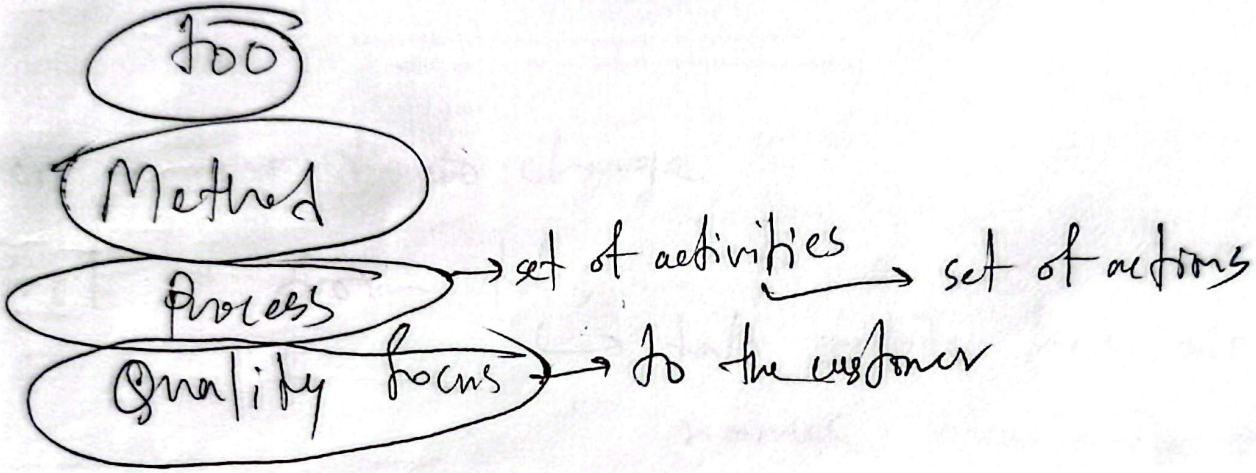
Companies: Service, Product and project based!

Reference: Software Engineering - A practitioner approach
— Roger pressman

Azizul Hakim

Long parameter list
Comment

SRS



Process:
1. Communication
2. Analysis
3. Design
4. Dev/coding
5. testing
6. Maintenance

Project: All of the activities including those which are not even directly related to the development.

Umbrella activities!

1. Understanding the problem
2. Plan a solution
3. Carry out the plan
4. Examine the result

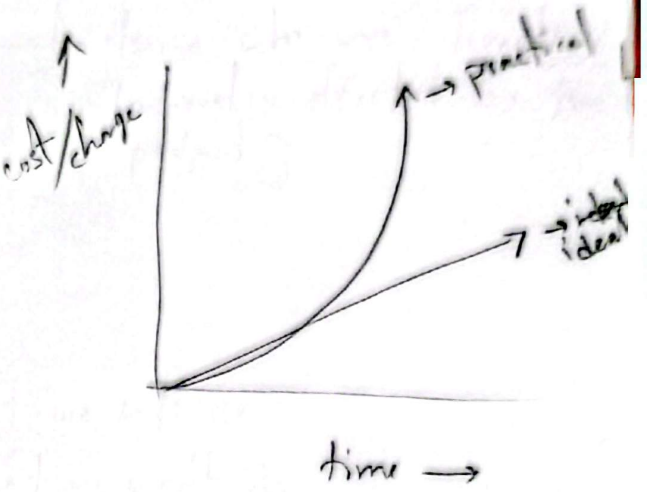
Principles:

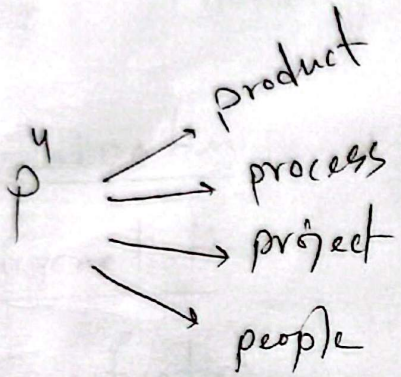
1. Reason it all exists
2. keep it ~~all~~ simple and stupid
3. Maintain the vision
4. what you ~~pro~~ produce others will consume
5. Be open to the future
6. Plan ahead ~~of~~ for reuse
7. Think

SRS

Process :

Activities
- communication
- Analysis
- Design / Modelling
- Develop
- Deployment





Requirement Engineering is a systematic approach to —

* what the customer needs
* Analysing need
* Assessing feasibility
* Negotiating a reasonable sol^n
* Specifying a sol^n → Defining sol^n with no ambig
  (with mathematics / diagram / chart)
* validating the spec
* Managing the requirement

Requirement Engi. Activities—

① Inception → start

output : ↳ defining set of stakeholders
↳ Analysing existing system

[stakeholders — those who are directly/ indirectly affected by the project]

② Elicitation → Enlight/Highlight/Review

output: → Requirement lists
↳ defining scope → ↳ what we will do
↳ what we won't do

③ Elaboration : * Describing Features

④ Negotiation :

⑤ Specification :

⑥ Validation :

⑦ Management :

Elaboration:

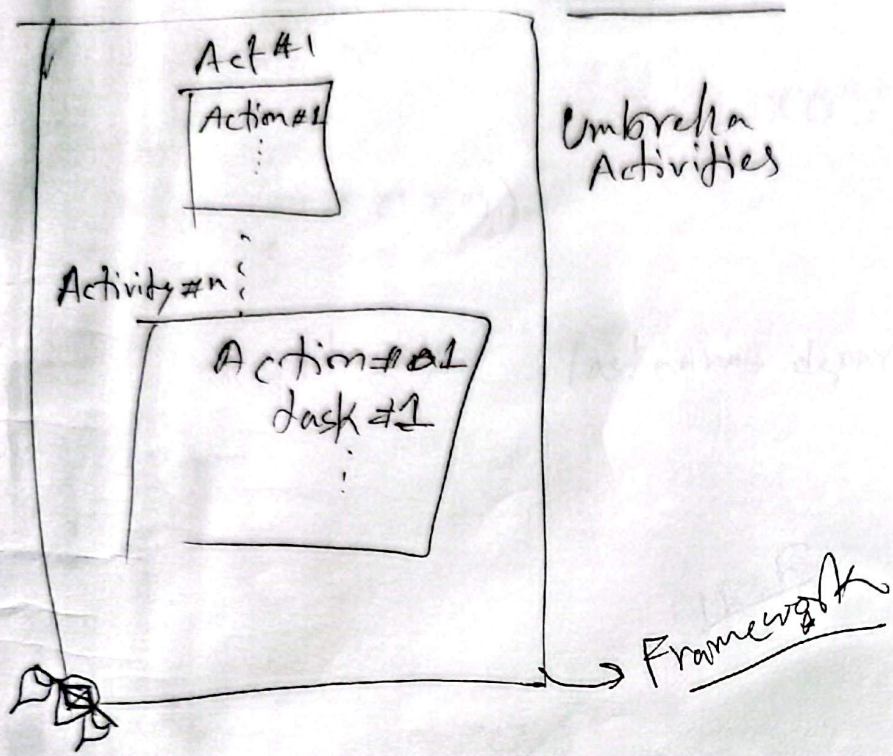
① Scenario based Modeling
- a. user/usage story
- b. Use-case diagram
- c. Activity diagram
- d. swimlane "

② Data based modeling — a. Data model with schema
b. E-R model

③ Class based Modeling — a. object determination
b. CRI model

④ Behavioural Modelling — a. Data flow Diagram
b. State transition "
c. Sequence "

SRS



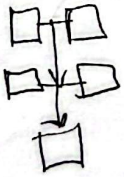
Umbrella Activities

uncertainty

Risk management
1. Change.
2. Estimation
3. Sizing

project management
* progress

Generic Process model : (Appropriate for different type software)

1. linear process flow model : Financial / Nuclear / Banking (Fail & proof type) - Real time system
2. Iterative :
3. Evolutionary →  : Requirements are not defined at first
4. Parallel →  very limited time ~~conflicts~~ constructs

Requirement Engi. Activities / SRS

Inception:

* study - search
* Discussing with stakeholders
* Identifying more stakeholders - Creating list of stakeholders
* Maybe a group of stakeholders - Each group having 1-2 people represent the whole group!

Recognizing ↑

* Considering multiple viewpoint of stakeholders.
* Clash! Resolve! Issues of viewpoint!
* Working toward collaboration with stakeholders.
* Teaming ↑

Output:

1. list of s.h.
2. Doc Diagram of existing system

First, context-free questions. → Preparing the questions beforehand

Then, related (contexted) ~~qp~~ question —

Elicitation:

* Scope:
  - What requirements cannot be automated
  - Revisit
  - list of requirement (QFD)

Another form of Requirement

1. functional Req.
2. Non-functional Req.

① Normal requirements: Directly told by customer

② Expected " : Necessary (even if not mentioned) to fulfill customer need.
- Scalability
- Security
- Maintainability

QFD: Quality Function Deployment

③ ~~Exc~~ Exciting Req: which are absolutely unnecessary for the business but will excite stakeholder.

→ Wow Factor!
→ Free
→ Marketting

Negotiation: → can be done any time
→ Resolving any conflict
→ Win-Win strategy

Specifcation: * The ultimate Document
* Output: →

Managemnt: * Change management

Validation: * How much confident I am for the requirement?
* Collaborate with Subject Expert

DBMS

* drop the dependent tables first.

select

* create

SRS

Company maturity : * Matured Team / defined Teams
*

↓
CMMI : (1-5) levels
↓
Maturity rating

Level-3 : Defined activities
" -4 : Measured " also
Level-5 : Optimized also

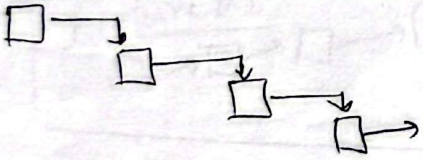
Metric: (Work maturity)
* # page
* # change
* Amount of time to change each ~~dis~~ takes

Prescriptive process model: (Not generic model)

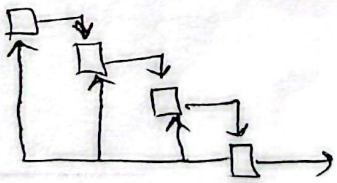
linear ~~proces flow~~ model's prescriptive models based on generics

① Waterfall model. ~~Disadvantages~~ * Features - * must have a clearly defined requirement, no change!
~~Mixed~~
* suitable for sensitive projects (like Financial, hard real time systems)

* Must have matured people.

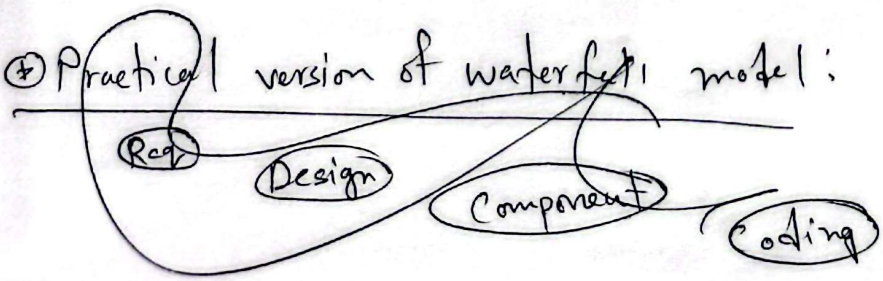


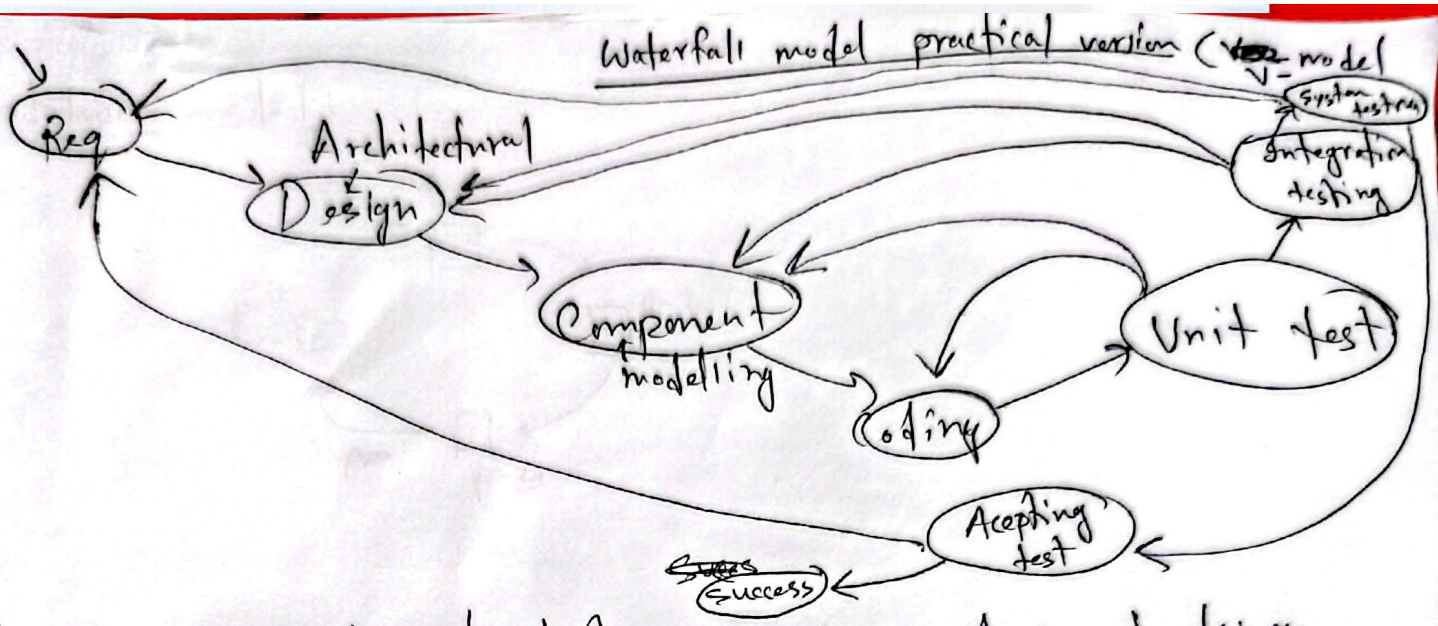
ideal
↳ no change



practical
↳ limited change
↳ change is expensive

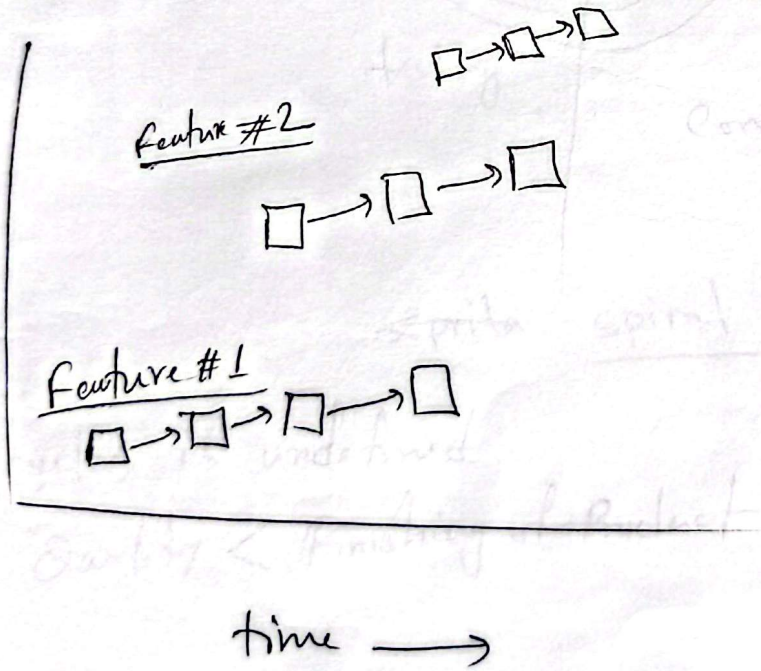
* Practical version of waterfall model:



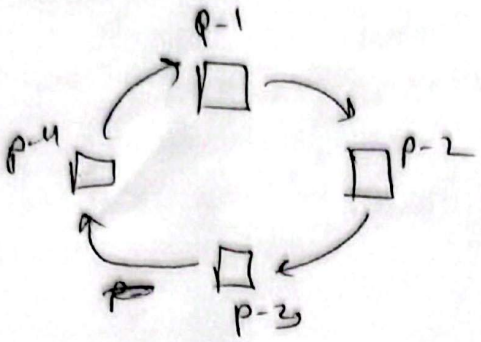


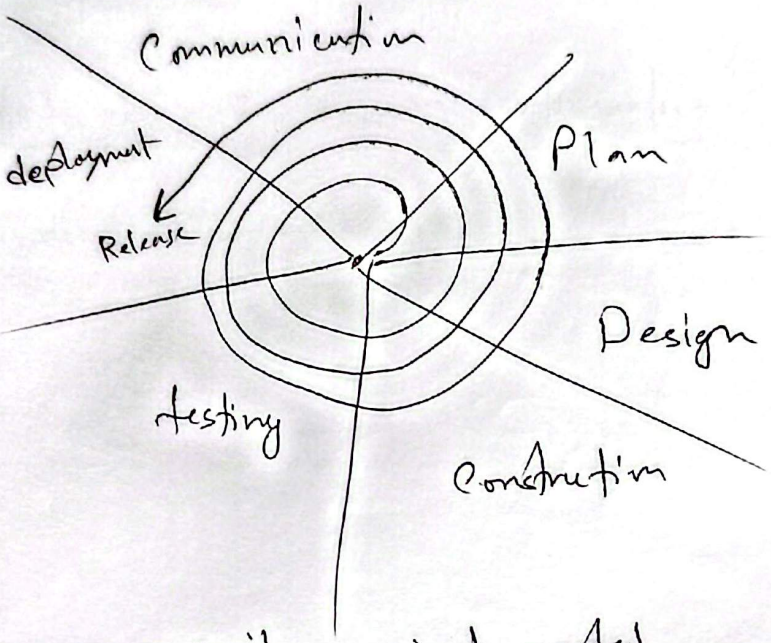
Unit testing — Statement testing, done by coders, checking logic on each statement

② Incremental process flow model



Prototype model:





Sprita spiral model

cycles is undefined

Quality < Finishing of Product

Parallel model: time constraint

* Inter activity parallism
* Intra activity actions parallelism

H.W.: Modern process flow model identification (Except agile)

Component Based Models

→ Component → Independently Product/Module runnable → reusable reusable

Compnt on the shelf ~~(COS)~~ **Cots**

* Experienced Model :
* Semi- " " :

Formal Methods : * using mathematics

Aspect Oriented Programming:

Inception

SRS

* Transcriptions for each stakeholders. with designation (not name) [Statement]

* A person can have multiple roles. but roles are counted independently.

* identifying company's core modules.

Elicidation

* Some modules cannot be automated.
* which are need to be automated
* Put importance on every stakeholder's viewpoint
* Arising conflict, then resolve

output – list of requirments

Usage Story (user's story) – Plain english / no technical talks

* Increasing understanding of my own on each step/activities.

SRS

* Software Engi. needs collaboration, adding more people in it can't increase productivity.

Spiral: Comprises - * cycle completion > Quality priority

! Change is inevitable:

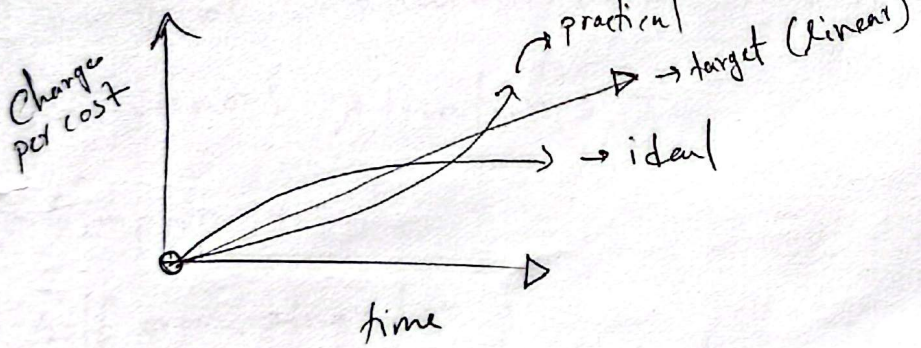
Agile → who accepts changes or welcomes changes, completes new task (changed).

↓

Software Process Model

↓

~~Good~~ Best model as everything is uncertain! tends to change.
But not appropriate where change is minimum.



Target: is to bring changes as early as possible to minimize cost → Philosophy of Agile model

~~Challenge~~ Challenges:

1. ~~It's~~ Software is difficult to predict
2. design and construction are interlinked
3. Software Eng. activities are not predictable as we may think.

Growth must be incremental, Adaptive to ATTACK
flexible Changle

↳ Definition of Agility

Agile Principles:

1. Satisfy the customer through early and continuous delivery
↳ to get changes early
2. Welcome changing requirements.
3. Deliver working software frequently → to get Feedback early
4. Business people and developers must work together.
5. Build projects around motivated people.
6. ~~Face~~ face to face conversation. ∵ Discouring formal communication
7. Working software is the primary measure of progress.
8. ~~Continuous~~ Continuous pace indefinitely.
9. Continuous Attention.
10. Simplicity
11. Self organizing team
12. Adjust behaviour accordingly.

Hello

Human Factors: 1. Competence
2. Common Focus → ability to bring everyone into a common focus
3. Collaboration. → To give ownership to team members of the project other
4. Decision-making ability → To gain experience
5. Fuzzy-problem solving ability
↳ indefinite solution

6. Mutual trust and Respect.

7. Self-organizing

MOI model → To be Leader

- Managment
- Organizing
- Innovation

factory dp

SRS

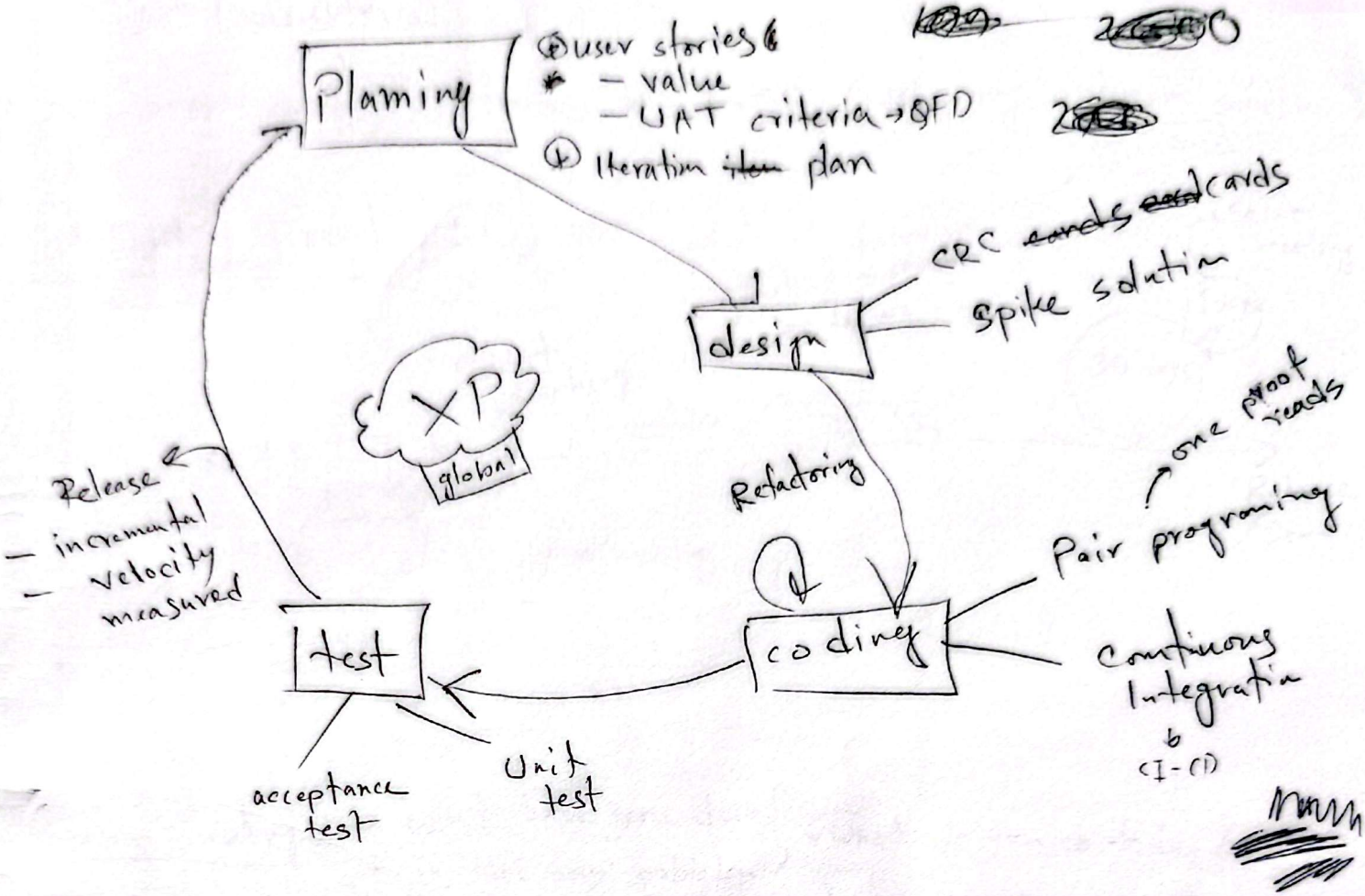
Agile → mid level companies follow

XP → extreme programming

(*) XP model : Model of Agile

Values:
1. communication → informal usually
2. Simplicity → design as currently required
3. feedback → customers, developers, software itself from
4. courage. → discipline
5. Respect → client can change requirement.

QFD → when: → creating user stories
↳ After delivery



Stories → short

Velocity → ~~Stories/cycle~~ : Pace measure
stories per cycle

* velocity ensures incremental growth

→ CRC → Class responsibility collaboration → Class based diagram
↳ single diagram in this model

* spike solution → complex

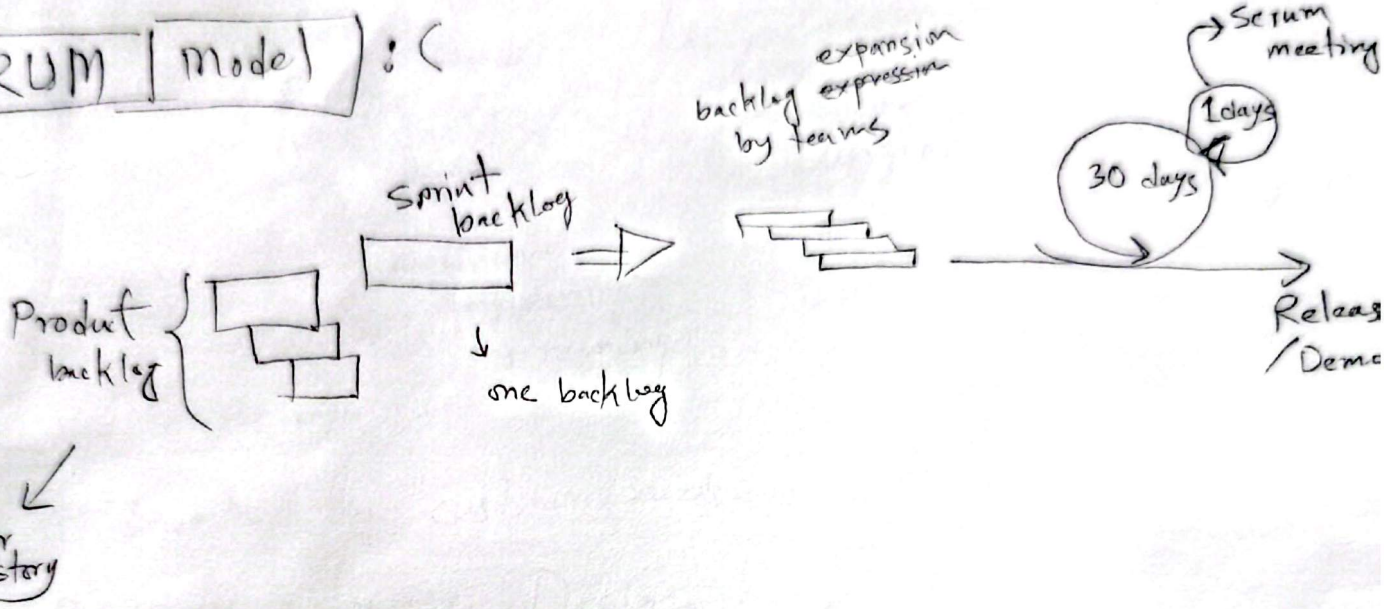
* Pair programming → ① coder ② Requirement engineer (~~junior~~)

~~features~~ - ① Require ~~rate~~ volatility
Debate ② Informal expression of requirement
③ Conflicting customer need
④ lack of formal design

IXP → Industrial XP

* XP compromises on these debates. (Not giving optimum solution)

SRUM Model :(



Scrum meeting: * what have you dwel? * What do you do today?
* is there any problelm?

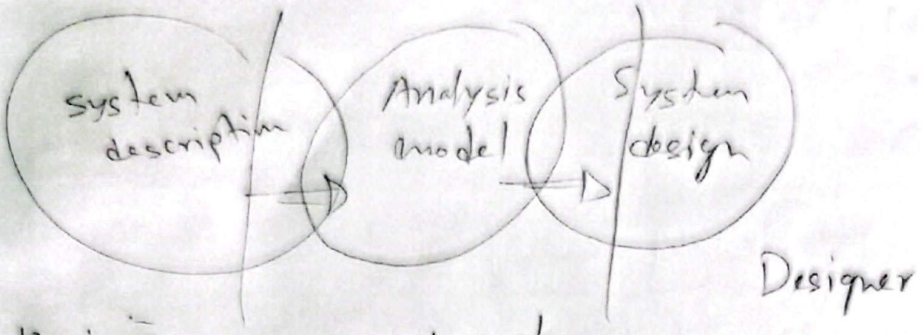
↓

~~Scrum~~
↳ Scrum master
↳ daily at beginning
↳ 10 - 15 mins

* used in our country !

1, 2, 3, 5 → Chapters

SRS



engineering

Objective for req. engi. :

1. To describe the customer needs.
2. Basis for software Design
3. To define a set of requirement for UAT

- The model should focus on requirement
- Each model should add overall understanding
- Minimize coupling
- Adds value to the stakeholders
- keep the model as simple as possible

Input source :

Technical literature (domain research)

Existing Application example

customer survey

expert advice

current and future requirement

Output ~~source~~ → (Deliverables)

User story —

QFD — Normal req
Expected req
Exciting "

Actor — interacts with the system
* puts input
* gets output
* or both
* can be a external system.

Primary actors —

Ultimate goal of QFD — UAT (user acceptance test)

Immediate goal —

1. User/usage story
* (to be exact)
* (scenario based model)
* (no technical term)
* easily reachable by client
* *

Secondary " :—

Actors are potential classes.

Problem domain ~~actor~~ —

Solution space ~~actor~~ —

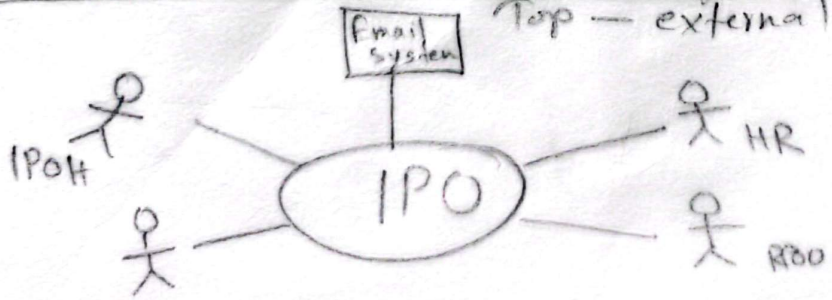
Se	Nam	P/S	Actor Pri/Sec
1			
2			

Description of use case
* naming it
* who are primary and secondary
* Activities/usage

→ (Full system)

level-0 - Use case modeling —

Top — external

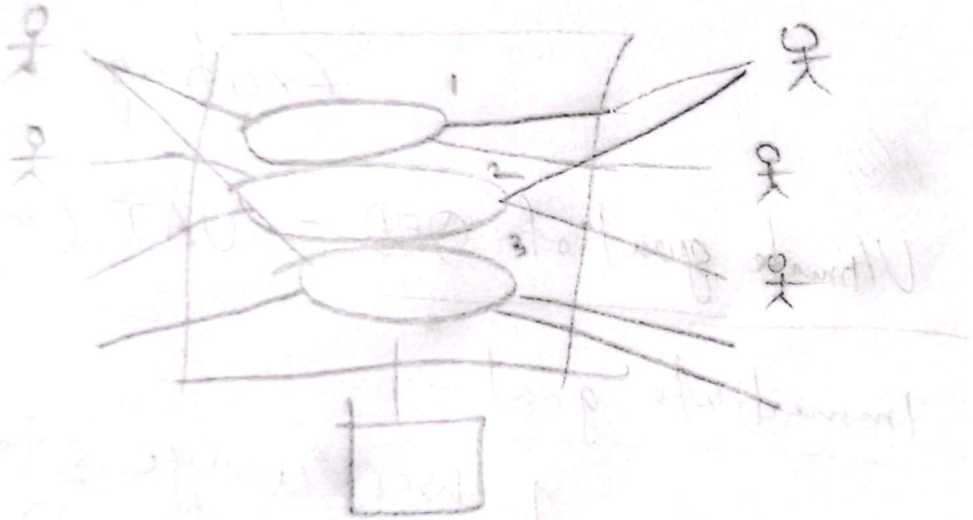


left side - primary

right - secondary

Level-1 use case — (multiple sub systems)

↳ substantial amount of work



level-1.2 / level-1.1

Assignment — 1. Use case diagram with description